SECTOR ARSENAL
SECTOR ARSENAL
October 1-2, 2025
METRO TORONTO CONVENTION CENTRE

Siege – Security Competitions for Defenders

Dmitriy Beryoza

What is this talk about?

We present a tool for building and hosting competitions that teach defensive cybersecurity skills.

About me

Dmitriy Beryoza https://www.linkedin.com/in/beryozad



- Principal Security Researcher at ***Vectra AI***
- Prior to that - Pentester/Secure Software Development Advocate with X-Force Ethical Hacking Team at ***IBM Security***
- 30+ years in software design and development
- Team lead/challenge builder for ***CyberSci***
- Presented at DEF CON Cloud Village, BSides SF/LV/Ottawa, Hackfest, OWASP meetups
- Ph.D. in Computer Science; OSCP, GREM, CISSP, and others
- *Interests*: reverse engineering, secure software development, CTF competitions

Agenda

- Why does one need to practice defensive security skills?
- What is a defence competition environment?
- Competition mechanics
- Architecture
- Setup
- Developing challenges
- Wrap-up

Software security does not improve

- Despite attention paid to cybersecurity, problems keep growing
  - CVE rates increase significantly y/y
    - 18% in 2025 [securityvulnerability.io]
  - Incident rates grow y/y
    - U.S. breaches increased 78% in 2023 over 2022 [WSJ]
- Why is this happening? Some of the reasons include:
  - Incentive misalignment
    - Vendors are rewarded for new features, usability, speed of delivery over security
  - Lack of liability
    - No legal or financial liability for vulnerabilities in software
  - Insufficient developer cybersecurity skills

Insufficient developer knowledge and skills

- I include in “developers” all builders: programmers, QA, architects, IT, DevOps
- Developers today have insufficient exposure to cyber knowledge
  - Most college-educated developers didn’t study cybersecurity in school
  - Lots of developers are self-taught
- Today CS education has improved, but there are still issues
  - Dedicated cybersecurity degrees exist but they are not for regular developers
  - In many CS programs cyber subjects are optional
- Practice is essential to having solid skills
  - Practicing to build secure software is harder than other software engineering skills
  - Experts with offensive security knowledge need to be integrated into the learning process

Available learning opportunities

- One would argue that there are lots of opportunities to learn practical security skills:
  - CTF (capture the flag) competitions (Jeopardy and A/D)
    - CTFTime.org (100s of CTFs/year); competitions at security conferences; internal company events, international competitions
  - Hack the Box, TryHackMe, Vuln Hub
  - Challenge sites (Wechall.net)
  - Practical security certificates (e.g. OSCP)
- Trouble is - all of these practice ***offensive*** security knowledge
  - These learning opportunities are not good match for training developers
  - Not relevant and/or intimidating – they exercise knowledge developers don't have and have no use for

Defence competitions

- Defence competitions & environments do exist but they are not nearly as common:
  - E.g. Locked Shields, Blue Team Village CTF, OpenBAS, etc.
  - Attack/Defence competitions have a defence component but you need deep offsec knowledge to participate
- I believe we need more accessible practical opportunities to learn defensive knowledge and skills
  - Such exercises would be appealing to wide range of builders (not just cyber professionals)
  - Universities, businesses, governments should find them useful to help train their personnel

Siege

- We releasing **Siege** – an open-source defence competition framework
  - Built as part of my work with CyberSci, a Canadian cybersecurity non-profit
  - We have used it to host defence-style competitions for the past 5 years
  - Hope it will be useful to anyone that needs to develop and defend software systems

- Software can be found at:

https://siege.software

https://github.com/0xd13a/siege



PSA: Canada's Cyber Security Challenge

- National cybersecurity competition for university students (running since 2010)
- Regional and national events (60+ teams), winners represent Canada internationally
- Team Canada competed in Prague, Athens, Vienna, San Diego, Santiago
- **Join us! Become a:**
- ***Sponsor*** - help us grow the next generation of cyber professionals
- ***Employer*** - interview and hire the brightest students in the field
- ***Organizer*** - join our team to organize events and build challenges
- ***Player*** - start or join your university team





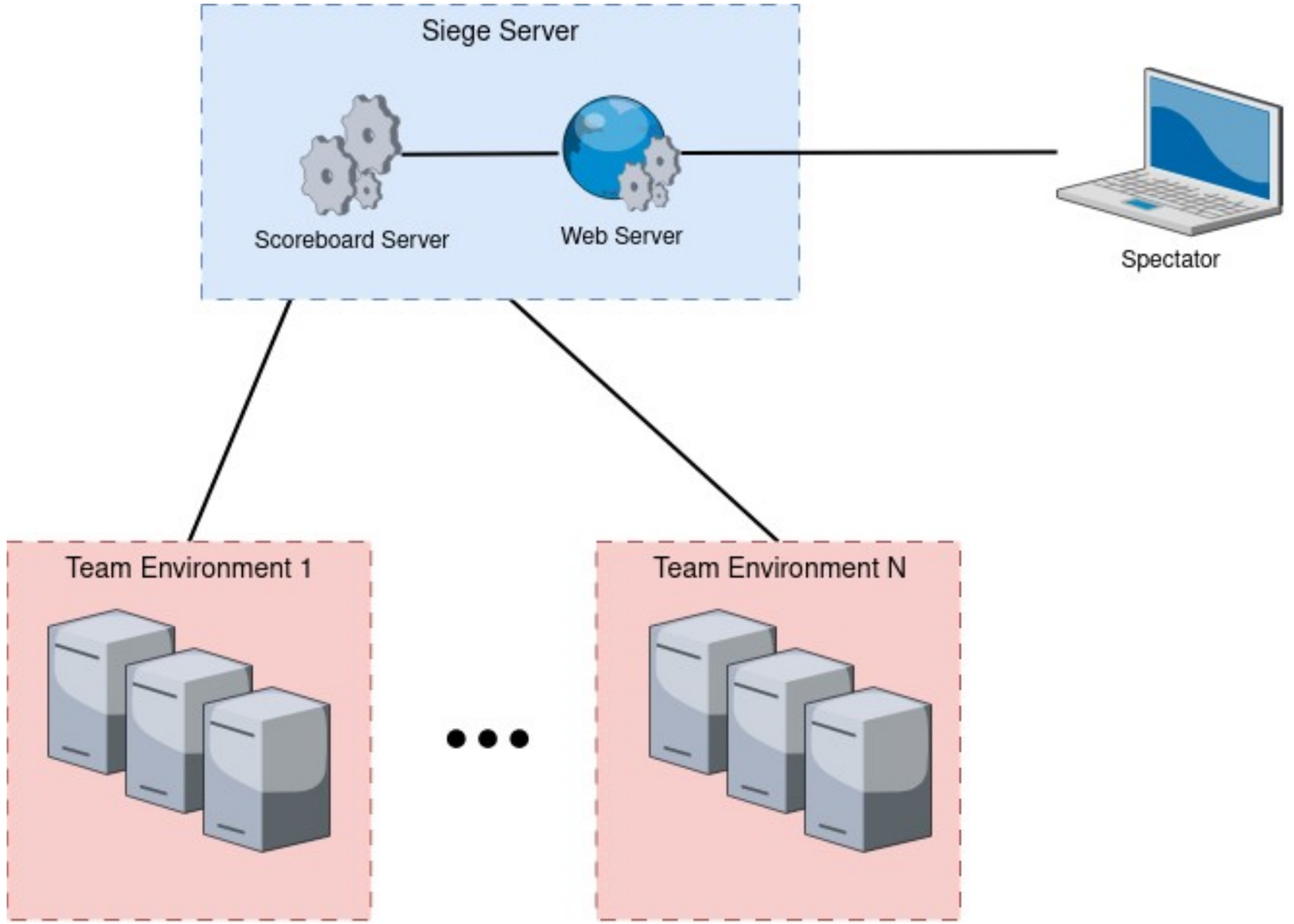
- https://cybersecuritychallenge.ca/

How does it work?

- During the competition teams have access to a set of vulnerable services
- Competition environment sends series of requests to those services
  - Malicious requests exploit the vulnerabilities
  - Benign requests make sure the service functionality is working as intended
- Players study services and the incoming traffic
  - They try to fix vulnerabilities to make malicious requests fail
  - Benign requests must continue working correctly to maintain service functionality
- Teams are awarded and penalized with points; the one with the highest score wins
- This exercise teaches valuable skills:
  - Code analysis
  - Vulnerability discovery and mitigation
  - Traffic analysis
  - Service uptime management

General architecture

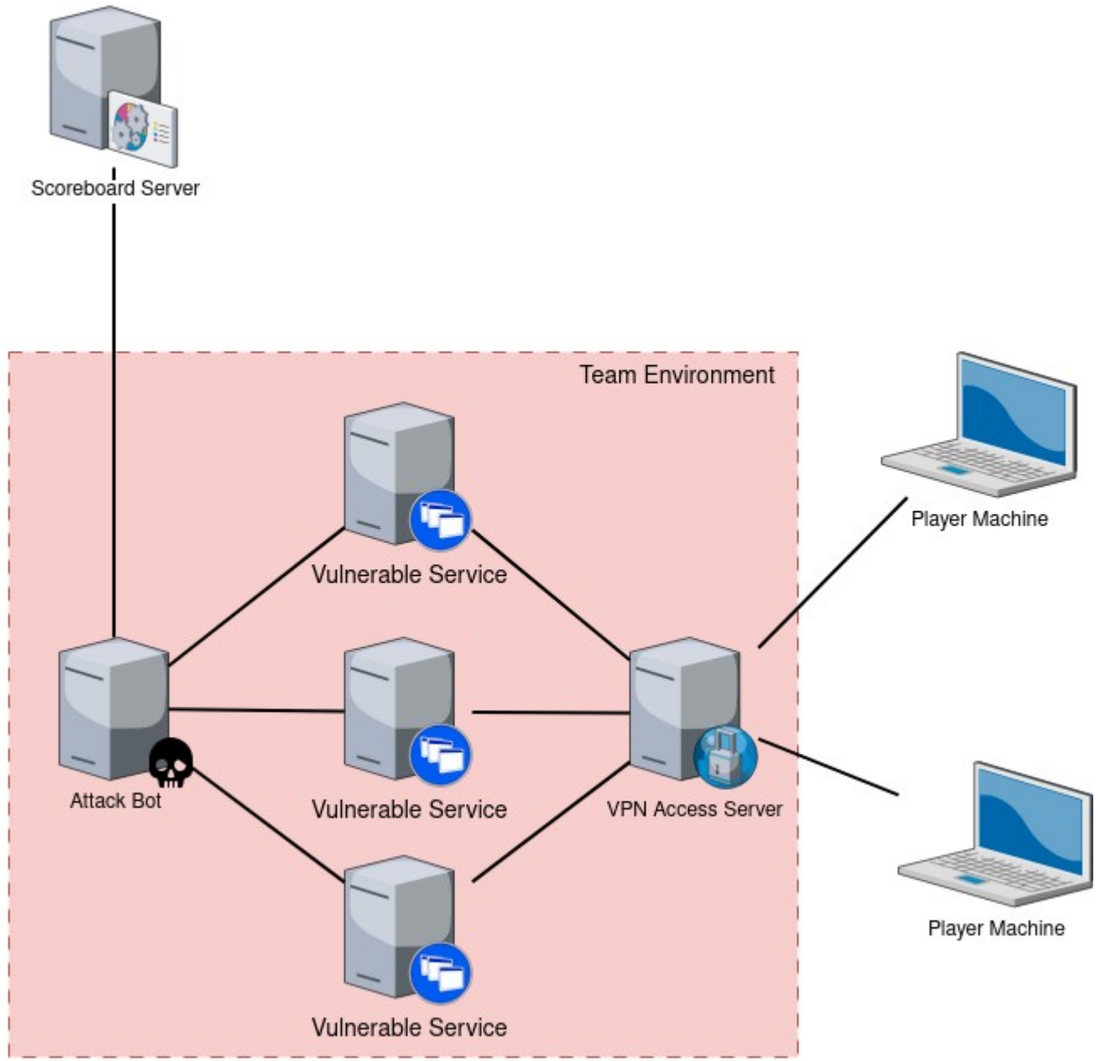
- *Siege server* (Python) controls the competition flow and manages the scoreboard
- *Web server* displays competition scoreboard and individual teams' progress
- Each team has an *environment* in which attacks execute and players fix service vulnerabilities



https://siege.software

Team environment

- Every team has a VPN access through a *VPN Access Server* host into a small network of VMs
- An *attacker bot* (Python) runs *attacker scripts*, one per service. Each executes series of benign and malicious requests against vulnerable service on the network. Bot reports results to Siege server
- Players have full control of several machines hosting vulnerable services and try to discover and fix security vulnerabilities



https://siege.software

Competition setup

- We provide scripts to manage infrastructure (Terraform/Ansible)
  - They build and bring up the scoreboard server and team environments
- Configuration files define competition settings, teams, and available services
  - We provide a sample configuration and a set of services to get you started
- On the eve of the competition
  - Environment is stood up
  - Settings and encryption keys are generated and distributed to secure communications between server and bots
  - VPN configurations and access instructions are distributed to teams (4-5 people per team)

Competition timeline

- Study period (T-1hr)
  - Teams are given access to their subnets
  - Study period is spent analyzing services and initiating fixes
- Competition clock starts (T+0h)
  - Competition is divided in a series of ***ticks*** (typically 2 minutes)
  - Every tick a series of requests, benign and malicious, are sent to the services
  - Teams capture incoming traffic, trying to determine what exploits are sent
  - Vulnerabilities are continuously fixed, services are rebuilt
- Competition ends (T+4h)
  - 4 hours seems to be a sweet spot to fix 3-5 services, longer may be tiring
  - Winners are determined
  - Resources are torn down

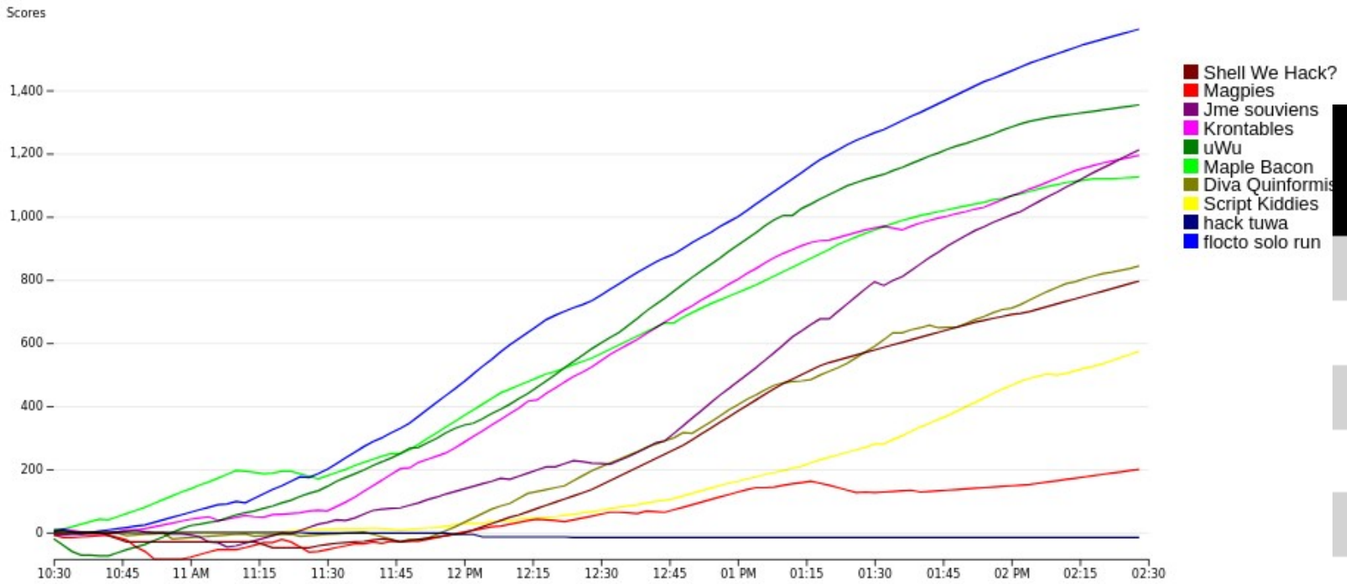
SECTOR ARSENAL

Scoreboard

CyberSci

CyberSci Nationals 2025

Competition has finished.



Time	Tick	Fuel Management Service						
		😇	😶	😈	😼	💀	🤓	Σ
5/31/2023, 11:43:00 PM	168	5			5			10
5/31/2023, 11:42:00 PM	167	5			5			10
5/31/2023, 11:41:00 PM	166	5			5			10
5/31/2023, 11:40:00 PM	165	4	-1		5			8
5/31/2023, 11:39:00 PM	164	5			5			10

Place	Team	Score
🥇 1	flocto solo run	1594
🥈 2	uWu	1354
🥉 3	Jme souviens	1212
4	Krontables	1194
5	Maple Bacon	1126

Scoring

- We usually score each request as follows (values are configurable):
  - Successful benign request - +1 point
  - Failed benign request - -1 point
  - Successful malicious request - -1 point
  - Failed malicious request - +1 point
  - Service down - -1 point
- We suggest sending an equal number of both types of requests per tick (e.g. 5 benign and 5 malicious)
  - This way the points for the service stay at 0 if nothing changes
- Request combinations can be configured with a schedule to add variability and unpredictability
- Scores can be capped to avoid “runaway” winners (a team finishing fixes early and accumulating positive points indefinitely)

Building vulnerable services

- Each service has a companion attack script developed, which sends benign and malicious requests
- Most services are available in source code, but some can require RE
- We usually host services on separate VMs to ease of management
  - Putting service in a container inside the VM facilitates building and deploying code
- Pretty much any technology can be used in a service:
  - Traditional website technologies
  - Common protocols (LDAP, IMAP, DNS, etc.)
  - Bespoke APIs
  - Terminal software
  - GUI apps
  - Misconfigured cloud services, environments, firewalls
  - AI chatbots

Best practices

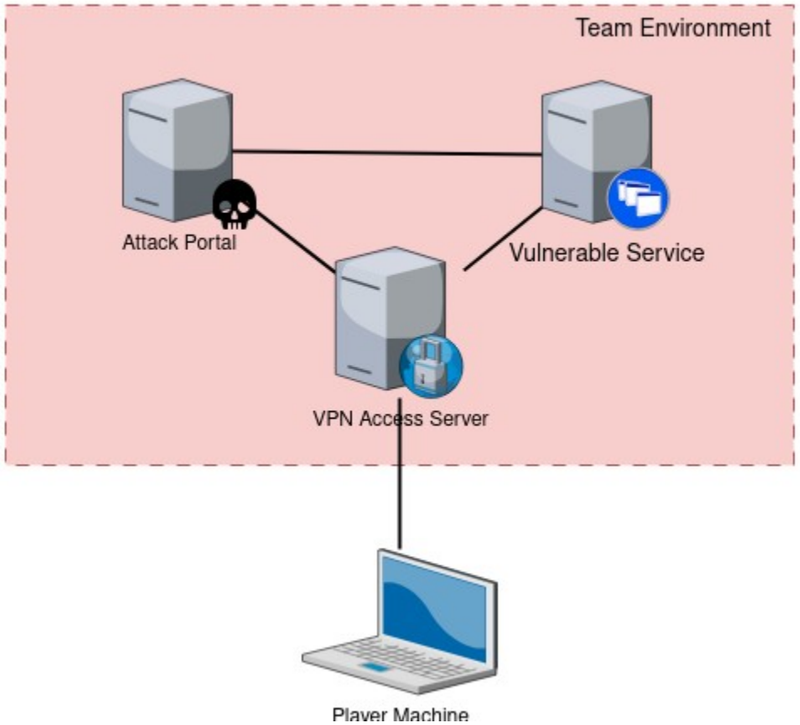
- Players will be tempted to block malicious requests (which is allowed but not ideal)
  - The preferred outcome is for players to fix the vulnerabilities
- Attacker functionality needs to be built so that requests are variable and contain no easily identifiable patterns
  - This will complicate building firewall rules to block exploits
- Encryption will also make blocking requests harder
- Interestingly, more advanced exploits (RCE, XSS, SQLi) are often easier to block
  - Logic errors may be less sophisticated but are harder to block

Best practices (continued)

- Good attack scripts should have some basic features:
  - Robustness in the face of any responses from the service
  - No chance of deadlock, timeouts, resource exhaustion
  - Requests must be guaranteed to execute in a short time to avoid tick overruns
  - Requests should be able to execute independent of each other
  - Request content must be as variable as possible to reduce possibility of blocking
  - Distinguishing between benign and malicious requests should be non-trivial

Siege Light

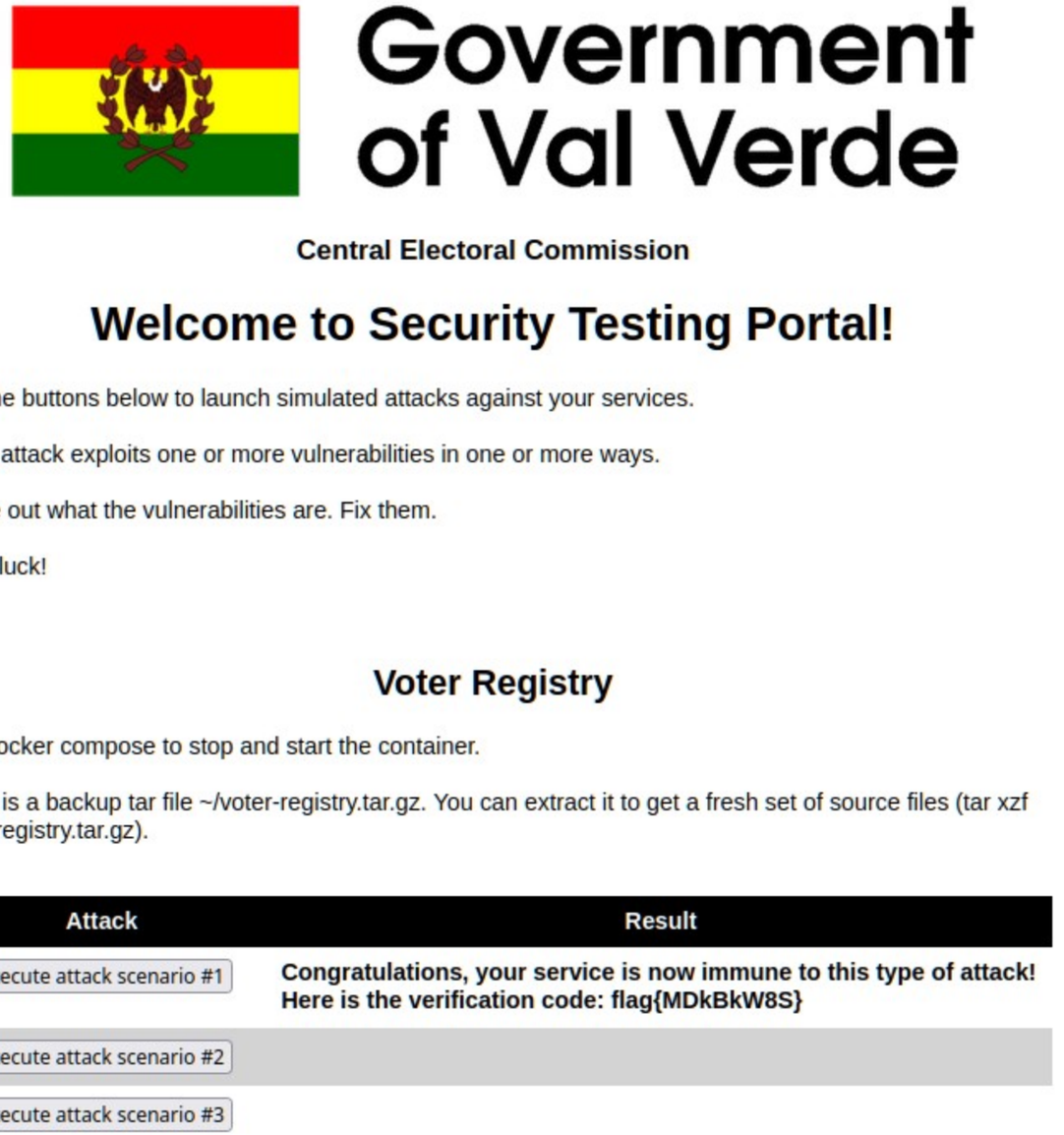
- In some cases building and running and entire competition is too much work
- We provide ability to run challenges in “light” mode that does not require a central scoreboard server or attack bots
- Teams get simple environments consisting of an attack portal machine and a set of vulnerable services
- Players drive attacks by interacting with an attack portal
- Attack is a series of requests, some of which exploit a particular vulnerability



https://siege.software

Siege Light (continued)

- Player analyzes requests and tries to fix the vulnerability
- If vulnerability is fixed, the attack fails and portal reveals a *flag*
- Flag can then be submitted to a traditional Jeopardy competition scoreboard (e.g. CTFd)



Government of Val Verde

Central Electoral Commission

Welcome to Security Testing Portal!

Use the buttons below to launch simulated attacks against your services.

Every attack exploits one or more vulnerabilities in one or more ways.

Figure out what the vulnerabilities are. Fix them.

Good luck!

Voter Registry

Use docker compose to stop and start the container.

There is a backup tar file ~/voter-registry.tar.gz. You can extract it to get a fresh set of source files (tar xzf voter-registry.tar.gz).

Attack	Result
Execute attack scenario #1	**Congratulations, your service is now immune to this type of attack! Here is the verification code: flag{MDkBkW8S}**
Execute attack scenario #2	
Execute attack scenario #3	

https://siege.software

What's next?

- Planned future work:
  - GUI for competition configuration and management
  - Support for a production-quality WSGI server like Gunicorn
  - Isolation for attacker scripts
  - AWS support for competition environment
  - Option to host some of the infrastructure in containers
- How to contribute?
  - Questions, ideas, bug fixes and new features are welcome!
  - Log and tag issues to communicate
  - For small fixes: submit a PR; maintain code style and test and document your changes
  - For bigger features: let's discuss first

Wrap-up

- Acknowledgments:
  - Big shout out to CyberSci crew that helped make this tool possible: **HapiRat, jacksimple, Fen-nix, trentholmes, willemgooderham, iamsilk, ApprenticeofEnder, crazyeights225**

Thank you for listening and please

reach out with any questions!

https://www.linkedin.com/in/beryozad/

https://siege.software

https://github.com/0xd13a/siege